Name : Deras Pangestu Gusti (08)
Class : SIB 1G
NIM : 254107060114

JOBSHEET 14
Tree

14.1 Learning Objectives

After completing this practicum, students will be able to:

1. Understand tree models, particularly binary trees.
2. Design and declare the structure of binary tree algorithms.
3. Apply and implement binary tree algorithms in the context of binary search trees.

14.2 Experiment 1

Implementation of Binary Search Tree Using Linked List (100 Minutes)

14.2.1 Labs Activity

In this experiment, a Binary Search Tree will be implemented using a linked list, focusing on its fundamental operations.

1. In the project created during the previous session, we created a new package named **Week14**.
2. Create the following classes:
   a. Student00.java
   b. Node00.java
   c. BinaryTree00.java
   d. BinaryTreeMain00.java

   Change 00 with your order number.
3. Implement Class Student00, Node00, BinaryTree00 based on the following class diagram:

Student
nim: String name: String className: String ipk: double
Student() Student (nm: String, name: String, kls: String, ipk: double) print(): void

Node
data: Student left: Node right: Node
Node (left: Node, data:Student, right:Node)

BinaryTree
root: Node

BinaryTree() add(data: Student): void

find(ipk: double) : boolean traversePreOrder (node : Node) : void traversePostOrder (node : Node) void traverseInOrder (node : Node): void getSuccessor (del: Node) delete(ipk: double): void

4. In the **Student00** class, declare the attributes according to the **Student** class diagram shown above. Also, add the constructor and methods as specified in the diagram.

```
public class Student00 {
    String nim, name, className;
    double ipk;

    public Student00(){
    }
    public Student00(String nm, String nama, String kls, double ip){
        nim = nm;
        name = nama;
        className = kls;
        ipk = ip;
    }
    void print(){
        System.out.println(nim+" - "+name+" - "+className+" - "+ipk);
    }
}
```

5. In the **Node00** class, add the attributes **data**, **left**, and **right**, along with both a default constructor and a parameterized constructor.

```
public class Node00 {
    Student00 data;
    Node00 left;
    Node00 right;

    Node00(){
    }
    Node00(Student00 data){
        this.data = data;
        left = null;
        left = null;
    }
}
```

6. In the **BinaryTree00** class, add the attribute **root**.

```
public class BinaryTree00 {
    Node00 root;
}
```

7. Add a constructor and the **isEmpty()** method in the **BinaryTree00** class.

```
    public BinaryTree00(){
        root = null;
```

```
    }

    public boolean isEmpty(){
        return root == null;
    }
```

8. Add the **add()** method in the **BinaryTree00 class**. Nodes should be inserted into the binary search tree based on the student's GPA (IPK) value. In this case, the node insertion process is implemented non-recursively to make the insertion flow easier to understand. However, a recursive approach would result in more efficient and concise code.

```
    public void add(Student00 data){
        if(isEmpty()){
            root = new Node00(data);
        } else {
            Node00 current = root;
            while(true){
                if(data.ipk < current.data.ipk){
                    if(current.left != null){
                        current = current.left;
                    } else {
                        current.left = new Node00(data);
                        break;
                    }
                } else if(data.ipk > current.data.ipk){
                    if(current.right != null){
                        current = current.right;
                    } else {
                        current.right = new Node00(data);
                        break;
                    }
                } else {
                    break;
                }
            }
        }
    }
```

9. Add **find()** method

```
    public boolean find(double ipk){
        boolean result = false;
        Node00 current = root;
        while(current != null){
            if(current.data.ipk == ipk){
                result = true;
                break;
            } else if(ipk < current.data.ipk){
                current = current.left;
            } else {
                current = current.right;
            }
        }
        return result;
    }
```

10. Add the methods **traversePreOrder(), traverseInOrder()**, and **traversePostOrder()** to the **BinaryTree00** class. These traversal methods are used to visit and display the nodes in the binary tree using pre-order, in-order, and post-order traversal modes, respectively

```
public void traversePreOrder(Node00 node){
    if(node != null){
        node.data.print();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}
public void traverseInOrder(Node00 node){
    if(node != null){
        traverseInOrder(node.left);
        node.data.print();
        traverseInOrder(node.right);
    }
}
public void traversePostOrder(Node00 node){
    if(node != null){
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.data.print();
    }
}
```

11. Add the method **getSuccessor().** This method will be used during the deletion process of a node that has two children.

```
Node00 getSuccessor(Node00 del){
    Node00 successor = del.right;
    Node00 successorParent = del;
    while(successor.left != null){
        successorParent = successor;
        successor = successor.left;
    }
    if(successor != del.right){
        successorParent.left = successor.right;
        successor.right = del.right;
    }
    return successor;
}
```

12. Add the **delete()** method. Within this method, include a check to determine whether the tree is empty; if it is not, proceed to locate the node to be deleted.

```
public void delete(double ipk){
    if(isEmpty()){
        System.out.println("Tree is empty!");
        return;
    }
    Node00 parent = root;
    Node00 current = root;
    boolean isLeftChild = false;
    while(current.data.ipk != ipk){
        parent = current;
```

```
            if(ipk < current.data.ipk){
                isLeftChild = true;
                current = current.left;
            } else {
                isLeftChild = false;
                current = current.right;
            }
            if(current == null){
                System.out.println("Couldn't find data!");
                return;
            }
        }
        //delete node with no children
        if(current.left == null && current.right == null){
            if(current == root){
                root = null;
            } else if(isLeftChild){
                parent.left = null;
            } else {
                parent.right = null;
            }
        } else if(current.right == null){//delete node with a left child
            if(current == root){
                root = current.left;
            } else if(isLeftChild){
                parent.left = current.left;
            } else {
                parent.right = current.left;
            }
        } else if(current.left == null){//delete node with a right child
            if(current  ==  root){
                root = current.right;
            } else if(isLeftChild){
                parent.left = current.right;
            } else {
                parent.right = current.right;
            }
        } else {//delete node with 2 children
            Node00 successor = getSuccessor(current);
            if(current == root){
                root = successor;
            } else if(isLeftChild){
                parent.left = successor;
            } else {
                parent.right = successor;
            }
            successor.left = current.left;
        }
    }
```

13. Open the **BinaryTreeMain00** class and add the **main**() method. Then, include the following code:

```
public class BinaryTreeMain00 {
    public static void main(String[] args) {
        BinaryTree00 bst = new BinaryTree00();
        bst.add(new Student00("244107020138", "Devin", "TI-1I", 3.57));
        bst.add(new Student00("244107020023", "Dewi", "TI-1I", 3.85));
```

```
        bst.add(new Student00("244107020225", "Wahyu", "TI-1I", 3.21));
        bst.add(new Student00("244107020076", "Angelina", "TI-1I", 3.54));

        System.out.println("Student list (in-order traversal)");
        bst.traverseInOrder(bst.root);

        System.out.println("Search data");
        System.out.print("Search a student with IPK: 3.54: ");
        String result = bst.find(3.54) ? "Found" : "Not Found";
        System.out.println(result);

        System.out.print("Search a student with IPK: 3.22: ");
        result = bst.find(3.22) ? "Found" : "Not Found";
        System.out.println(result);

        bst.add(new Student00("244107020223", "Andhika", "TI-1I", 3.72));
        bst.add(new Student00("244107020226", "Bima", "TI-1I", 3.37));
        bst.add(new Student00("244107020181", "Eiyu", "TI-1I", 3.46));
        System.out.println("Student list:");
        System.out.println("In-order traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println("Pre-order traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println("Post-order traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println("Data deletion");
        bst.delete(3.57);
        System.out.println("Student list after deletion:");
        bst.traverseInOrder(bst.root);
    }
}
```

14. Compile and run the **BinaryTreeMain00** class to simulate the execution of the implemented binary tree program.

```
1   package com.jobsheet14;
2
3   public class Student8
4   {
5       String  nim,
6               name,
7               className;
8       double  ipk;
9
10      public Student8() {}
11
12      public Student8 (String nim, String name, String className, double ipk)
13      {
14          this.nim = nim;
15          this.name = name;
16          this.className = className;
17          this.ipk = ipk;
18      }
19
20      void print()
21      {
22          System.out.println(nim + " - " + name + " - " + className + " - " + ipk);
23      }
24  }
```

```java
package com.jobsheet14;

public class Node8
{
    Student8    data;
    Node8       left, right;

    public Node8() {}

    public Node8(Student8 data)
    {
        this.data = data;
        left = null;
        right = null;
    }
}
```

```java
package com.jobsheet14;

public class BinaryTree8
{
    Node8 root;

    BinaryTree8()
    {
        root = null;
    }

    public boolean isEmpty()
    {
        return root == null;
    }

    public void add (Student8 data)
    {
        if (isEmpty())
            root = new Node8(data);
        else
        {
            Node8 current = root;

            while (true)
            {
                if (data.ipk < current.data.ipk)
                {
                    if (current.left != null)
                        current = current.left;
                    else
                    {
                        current.left = new Node8(data);
                        break;
                    }
                }
                else if (data.ipk > current.data.ipk)
                {
                    if (current.right != null)
                        current = current.right;
                    else
                    {
                        current.right = new Node8(data);
                        break;
                    }
                }
                else
                    break;
            }
        }
    }

```

```java
public boolean find (double ipk)
{
    boolean result = false;
    Node8 current = root;

    while (current != null)
    {
        if (current.data.ipk == ipk)
        {
            result = true;
            break;
        }
        else if (ipk < current.data.ipk)
            current = current.left;
        else
            current = current.right;
    }

    return result;
}

public void traversePreOrder (Node8 node)
{
    if (node != null)
    {
        node.data.print();
        traversePreOrder(node.left);
        traversePreOrder(node.right);
    }
}

public void traverseInOrder (Node8 node)
{
    if (node != null)
    {
        traverseInOrder(node.left);
        node.data.print();
        traverseInOrder(node.right);
    }
}

public void traversePostOrder (Node8 node)
{
    if (node != null)
    {
        traversePostOrder(node.left);
        traversePostOrder(node.right);
        node.data.print();
    }
}

```

```java
    Node8 getSuccessor (Node8 del)
    {
        Node8 successor = del.right;
        Node8 successorParent = del;

        while (successor.left != null)
        {
            successorParent = successor;
            successor = successor.left;
        }

        if (successor != del.right)
        {
            successorParent.left = successor.right;
            successor.right = del.right;
        }

        return successor;
    }

    public void delete (double ipk)
    {
        if (isEmpty())
        {
            System.out.println(x: "Tree is Empty!");
            return;
        }

        Node8 parent = root;
        Node8 current = root;
        boolean isLeftChild = false;

        while (current.data.ipk != ipk)
        {
            parent = current;

            if (ipk < current.data.ipk)
            {
                isLeftChild = true;
                current = current.left;
            }
            else
            {
                isLeftChild = false;
                current = current.right;
            }

            if (current == null)
            {
                System.out.println(x: "Couldn't find data!");
                return;
            }
        }

```

```
158         if (current.left == null && current.right == null)
159         {
160             if (current == root)
161                 root = null;
162             else if (isLeftChild)
163                 parent.left = null;
164             else
165                 parent.right = null;
166         }
167         else if (current.right == null)
168         {
169             if (current == root)
170                 root = current.left;
171             else if (isLeftChild)
172                 parent.left = current.left;
173             else
174                 parent.right = current.left;
175         }
176         else if (current.left == null)
177         {
178             if (current == root)
179                 root = current.right;
180             else if (isLeftChild)
181                 parent.left = current.right;
182             else
183                 parent.right = current.right;
184         }
185         else
186         {
187             Node8 successor = getSuccessor(current);
188 
189             if (current == root)
190                 root = successor;
191             else if (isLeftChild)
192                 parent.left = successor;
193             else
194                 parent.right = successor;
195 
196             successor.left = current.left;
197         }
198     }
199 }
```

```java
package com.jobsheet14;

public class BinaryTreeMain8
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        BinaryTree8 bst = new BinaryTree8();

        bst.add(new Student8(nim: "244107020138", name: "Devin", className: "TI-1I", ipk: 3.57));
        bst.add(new Student8(nim: "244107020023", name: "Dewi", className: "TI-1I", ipk: 3.85));
        bst.add(new Student8(nim: "244107020225", name: "Wahyu", className: "TI-1I", ipk: 3.21));
        bst.add(new Student8(nim: "244107020076", name: "Angelina", className: "TI-1I", ipk: 3.54));

        System.out.println(x: "Student list (in-order traversal)");
        bst.traverseInOrder(bst.root);

        System.out.println(x: "Search data");
        System.out.print(s: "Search a student with IPK: 3.54: ");
        String result = bst.find(ipk: 3.54) ? "Found" : "Not Found";
        System.out.println(result);

        System.out.print(s: "Search a student with IPK: 3.22: ");
        result = bst.find(ipk: 3.22) ? "Found" : "Not Found";
        System.out.println(result);

        bst.add(new Student8(nim: "244107020223", name: "Andhika", className: "TI-1I", ipk: 3.72));
        bst.add(new Student8(nim: "244107020226", name: "Bima", className: "TI-1I", ipk: 3.37));
        bst.add(new Student8(nim: "244107020181", name: "Eiyu", className: "TI-1I", ipk: 3.46));

        System.out.println(x: "Student list:");
        System.out.println(x: "In-order traversal:");
        bst.traverseInOrder(bst.root);
        System.out.println(x: "Pre-order traversal:");
        bst.traversePreOrder(bst.root);
        System.out.println(x: "Post-order traversal:");
        bst.traversePostOrder(bst.root);

        System.out.println(x: "Data deletion");
        bst.delete(ipk: 3.57);
        System.out.println(x: "Student list after deletion:");
        bst.traverseInOrder(bst.root);
    }
}
```

```
Student list (in-order traversal)
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020023 - Dewi - TI-1I - 3.85
Search data
Search a student with IPK: 3.54: Found
Search a student with IPK: 3.22: Not Found
Student list:
In-order traversal:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
Pre-order traversal:
244107020138 - Devin - TI-1I - 3.57
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020023 - Dewi - TI-1I - 3.85
244107020223 - Andhika - TI-1I - 3.72
Post-order traversal:
244107020181 - Eiyu - TI-1I - 3.46
244107020226 - Bima - TI-1I - 3.37
244107020076 - Angelina - TI-1I - 3.54
244107020225 - Wahyu - TI-1I - 3.21
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
244107020138 - Devin - TI-1I - 3.57
Data deletion
Student list after deletion:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
```

15. Observe and analyze the output generated during the program run.

```
Student list (in-order traversal)
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020023 - Dewi - TI-1I - 3.85
Search data
Search a student with IPK: 3.54: Found
Search a student with IPK: 3.22: Not Found
Student list:
In-order traversal:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
Pre-order traversal:
244107020138 - Devin - TI-1I - 3.57
244107020225 - Wahyu - TI-1I - 3.21
244107020076 - Angelina - TI-1I - 3.54
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020023 - Dewi - TI-1I - 3.85
244107020223 - Andhika - TI-1I - 3.72
```

```
244107020226 - Bima - TI-1I - 3.37
244107020076 - Angelina - TI-1I - 3.54
244107020225 - Wahyu - TI-1I - 3.21
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
244107020138 - Devin - TI-1I - 3.57



Data deletion
Student list after deletion:
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
```

14.2.2 Questions

1. Why is data search in a binary search tree more efficient compared to a regular binary tree?

   Because a binary search tree (BST) stores data in a sorted manner. Smaller values on the left, larger on the right allowing the search to follow a logical path and skip unnecessary nodes. A regular binary tree lacks this ordering, so every node might need to be checked.
2. What are the purposes of the **left** and **right** attributes in the **Node** class?
   a. *left → points to the node containing smaller data.*
   b. *right → points to the node containing larger data.*
3. a. What is the function of the **root** attribute in the **BinaryTree** class?
   It acts as the entry point or topmost node of the tree — all operations (search, traversal, insertion, deletion) begin from the root.
   b. When a **BinaryTree** object is first created, what is the initial value of **root**?
   It is null.
4. When the tree is empty and a new node is to be added, what process takes place?
   The new node becomes the root node. The program checks isEmpty(), and if true, assigns root = new Node00(data).
5. Consider the following line of code inside the **add**() method. Explain in detail the purpose of this line of code.

```
if(data.ipk < current.data.ipk){
     if(current.left != null){
         current = current.left;
     } else {
         current.left = new Node00(data);
         break;
     }
} else if(data.ipk > current.data.ipk){
     if(current.right != null){
         current = current.right;
     } else {
         current.right = new Node00(data);
         break;
     }
}
```

This code ensures the new node is inserted in the correct position based on the student's GPA (ipk). If the new data's ipk is smaller, the algorithm moves left; if the left child is empty, it creates

a new node there. It maintains the BST property — smaller values go left, larger go right.

6. Explain the steps involved in the **delete**() method when removing a node that has two children. How does the **getSuccessor**() method assist in this process?

    1. *Find the node to delete.*
    2. *Call getSuccessor() to locate the smallest node in the right subtree (the next higher value).*
    3. *Replace the deleted node's data with the successor's data.*
    4. *Adjust links so the successor's original position is correctly removed.*

14.3 Experiment 2

Implementation of Binary Tree Using Array (45 Minutes)

14.3.1 Labs Activity

1. In this binary tree implementation experiment using an array, the tree data is stored directly in an array and initialized within the **main**() method. Subsequently, the **in-order** traversal process will be simulated.
2. Create the classes **BinaryTreeArray00** and **BinaryTreeArrayMain00**, replacing **00** with your attendance number.
3. In the **BinaryTreeArray00** class, create the attributes **data** and **idxLast**. Also, implement the methods **populateData**() and **traverseInOrder**().

```
public class BinaryTreeArray00 {
    Student00[] data;
    int idxLast;
    public BinaryTreeArray00(){
        data = new Student00[10];
        idxLast = -1;
    }
    void populateData(Student00[] data, int idxLast){
        this.data = data;
        this.idxLast = idxLast;
    }
    void traverseInOrder(int idxStart){
        if(idxStart <= idxLast){
            if(data[idxStart] != null){
                traverseInOrder(2 * idxStart + 1);
                data[idxStart].print();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }
}
```

4. Then, in the **BinaryTreeArrayMain00** class, create the **main**() method and insert the code shown in the following image inside the method.

```
public class BinaryTreeArrayMain00 {
    public static void main(String[] args) {
        BinaryTreeArray00 bta = new BinaryTreeArray00();
        Student00 m1 = new Student00("244107020138", "Devin", "TI-1I", 3.57);
        Student00 m2 = new Student00("244107020023", "Dewi", "TI-1I", 3.85);
        Student00 m3 = new Student00("244107020225", "Wahyu", "TI-1I", 3.21);
        Student00 m4 = new Student00("244107020076", "Angelina", "TI-1I", 3.54);
        Student00 m5 = new Student00("244107020223", "Andhika", "TI-1I", 3.72);
        Student00 m6 = new Student00("244107020226", "Bima", "TI-1I", 3.37);
        Student00 m7 = new Student00("244107020181", "Eiyu", "TI-1I", 3.46);

        Student00[] data = {m1, m2, m3, m4, m5, m6, m7};
        bta.populateData(data, data.length-1);
        System.out.println("In-order traversal:");
        bta.traverseInOrder(0);
    }
}
```

5. Run the **BinaryTreeArrayMain00** class and observe the output!

```
In-order traversal:
244107020076 - Angelina - TI-1I - 3.54
244107020023 - Dewi - TI-1I - 3.85
244107020223 - Andhika - TI-1I - 3.72
244107020138 - Devin - TI-1I - 3.57
244107020226 - Bima - TI-1I - 3.37
244107020225 - Wahyu - TI-1I - 3.21
244107020181 - Eiyu - TI-1I - 3.46
```

```java
package com.jobsheet14;

public class BinaryTreeArray8
{
    Student8[] data;
    int idxLast;

    public BinaryTreeArray8()
    {
        data = new Student8[10];
        idxLast = -1;
    }

    void populateData (Student8[] data, int idxLast)
    {
        this.data = data;
        this.idxLast = idxLast;
    }

    void traverseInOrder (int idxStart)
    {
        if (idxStart <= idxLast)
        {
            if (data[idxStart] != null)
            {
                traverseInOrder(2 * idxStart + 1);
                data[idxStart].print();
                traverseInOrder(2 * idxStart + 2);
            }
        }
    }

}
```

```java
package com.jobsheet14;

public class BinaryTreeArrayMain8
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        BinaryTreeArray8 bta = new BinaryTreeArray8();

        Student8 m1 = new Student8(nim: "244107020138", name: "Devin", className: "TI-1I", ipk: 3.57);
        Student8 m2 = new Student8(nim: "244107020023", name: "Dewi", className: "TI-1I", ipk: 3.85);
        Student8 m3 = new Student8(nim: "244107020225", name: "Wahyu", className: "TI-1I", ipk: 3.21);
        Student8 m4 = new Student8(nim: "244107020076", name: "Angelina", className: "TI-1I", ipk: 3.54)
        Student8 m5 = new Student8(nim: "244107020223", name: "Andhika", className: "TI-1I", ipk: 3.72);
        Student8 m6 = new Student8(nim: "244107020226", name: "Bima", className: "TI-1I", ipk: 3.37);
        Student8 m7 = new Student8(nim: "244107020181", name: "Eiyu", className: "TI-1I", ipk: 3.46);

        Student8[] data = {m1, m2, m3, m4, m5, m6, m7};
        bta.populateData(data, data.length - 1);
        System.out.println(x: "In-order traversal:");
        bta.traverseInOrder(idxStart: 0);
    }
}
```

```
In-order traversal:
244107020076 - Angelina - TI-1I - 3.54
244107020023 - Dewi - TI-1I - 3.85
244107020223 - Andhika - TI-1I - 3.72
244107020138 - Devin - TI-1I - 3.57
244107020226 - Bima - TI-1I - 3.37
244107020225 - Wahyu - TI-1I - 3.21
244107020181 - Eiyu - TI-1I - 3.46
```

14.3.2 Questions

1. What is the purpose of the **data** and **idxLast** attributes in the **BinaryTreeArray** class?

a. *data: an array storing the tree's nodes.*
b. *idxLast: keeps track of the index of the last element in the array, showing how many nodes are currently in the tree.*

2. What is the function of the **populateData**() method?
   It initialises the tree by copying an existing array of nodes and setting idxLast to the last valid index.
3. What is the purpose of the **traverseInOrder**() method?
   It performs an in-order traversal, visiting the left child, then the current node, then the right child to display the tree's data in sorted order.
4. If a binary tree node is stored at index **2** in the array, at which indices are its left and right children located, respectively?
   a. *Left child* $\rightarrow$ *index* $2 \times 2 + 1 = 5$
   b. *Right child* $\rightarrow$ *index* $2 \times 2 + 2 = 6$
5. What is the purpose of the statement int **idxLast = 6** in Experiment 2, step 4?
   It indicates that the last valid index in the array is 6, meaning there are seven elements (indices 0–6) in the tree.
6. Why are the **indices 2 * idxStart + 1** and **2 * idxStart + 2** used in the recursive calls, and how do they relate to the structure of a binary tree represented as an array?
   For any node at index i,
   a. *Left child* $\rightarrow 2i + 1$
   b. *Right child* $\rightarrow 2i + 2$ *They maintain the logical parent–child relationship within the array structure.*

a. Assignments

Times Allocated: 150 minutes

1. Implement a recursive method **addRekursif**() in the **BinaryTree00** class to add nodes recursively.

```
200     public Node8 addRekursif(Node8 current, Student00 data)
201     {
202         if (current == null)
203             return new Node8(data);
204
205         if (data.ipk < current.data.ipk)
206             current.left = addRekursif(current.left, data);
207         else if (data.ipk > current.data.ipk)
208             current.right = addRekursif(current.right, data);
209
210         return current;
211     }
```

```
43      bst.root = bst.addRekursif(bst.root, new Student8(nim: "244107020138", name: "Devin", className: "TI-1I", ipk: 3.57));
44      bst.root = bst.addRekursif(bst.root, new Student8(nim: "244107020023", name: "Dewi", className: "TI-1I", ipk: 3.85));
45      bst.root = bst.addRekursif(bst.root, new Student8(nim: "244107020225", name: "Wahyu", className: "TI-1I", ipk: 3.21));
46      bst.root = bst.addRekursif(bst.root, new Student8(nim: "244107020076", name: "Angelina", className: "TI-1I", ipk: 3.54
47
48      System.out.println(x: "--- Student List (In-Order) ---");
49      bst.traverseInOrder(bst.root);
```

```
--- Student List (In-Order) ---
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
```

2. Create methods **getMinIPK**() and **getMaxIPK**() in the **BinaryTree00** class to retrieve and display the student data with the lowest and highest GPA(IPK) values stored in the binary search tree.

```
213     public void getMinIPK()
214     {
215         if (isEmpty())
216         {
217             System.out.println(x: "Tree is empty!");
218             return;
219         }
220 
221         Node8 current = root;
222 
223         while (current.left != null)
224             current = current.left;
225 
226         System.out.println(x: "Student with Minimum IPK:");
227         current.data.print();
228     }
229 
230     public void getMaxIPK()
231     {
232         if (isEmpty())
233         {
234             System.out.println(x: "Tree is empty!");
235             return;
236         }
237 
238         Node8 current = root;
239 
240         while (current.right != null)
241             current = current.right;
242 
243         System.out.println(x: "Student with Maximum IPK:");
244         current.data.print();
245     }
```

```
51          bst.getMinIPK();
52          bst.getMaxIPK();
53          System.out.println();
```

```
Student with Minimum IPK:
244107020225 - Wahyu - TI-1I - 3.21
Student with Maximum IPK:
244107020023 - Dewi - TI-1I - 3.85
```

3. Develop a method **displayStudentsWithIPKAbove** (**double threshold**) in the **BinaryTree00** class to display student data whose GPA(IPK) exceeds a specified threshold (e.g., above 3.50) within the binary search tree.

```java
public void displayStudentsWithIPKAbove(Node8 node, double threshold)
{
    if (node != null)
    {
        displayStudentsWithIPKAbove(node.left, threshold);

        if (node.data.ipk > threshold)
            node.data.print();

        displayStudentsWithIPKAbove(node.right, threshold);
    }
}
```

```java
bst.displayStudentsWithIPKAbove(bst.root, threshold: 3.50);
```

```
244107020076 - Angelina - TI-1I - 3.54
244107020138 - Devin - TI-1I - 3.57
244107020223 - Andhika - TI-1I - 3.72
244107020023 - Dewi - TI-1I - 3.85
```

4. Modify the **BinaryTreeArray00** class by adding the following methods:
   - **add(Student data)** to insert data into the binary tree represented as an array.
   - **traversePreOrder()** to perform a pre-order traversal of the binary tree.

```java
public void add(Student8 newData)
{
    if (idxLast == -1)
    {
        data[0] = newData;
        idxLast = 0;
        return;
    }

    int currentIdx = 0;

    while (true)
    {
        if (newData.ipk < data[currentIdx].ipk)
        {
            int leftChildIdx = 2 * currentIdx + 1;

            if (leftChildIdx >= data.length)
            {
                System.out.println(x: "Array capacity exceeded!");
                break;
            }

            if (data[leftChildIdx] == null)
            {
                data[leftChildIdx] = newData;
                if (leftChildIdx > idxLast) idxLast = leftChildIdx;
                break;
            }
            else
                currentIdx = leftChildIdx;
        }
```

```
            else if (newData.ipk > data[currentIdx].ipk)
            {
                int rightChildIdx = 2 * currentIdx + 2;

                if (rightChildIdx >= data.length)
                {
                    System.out.println(x: "Array capacity exceeded!");
                    break;
                }

                if (data[rightChildIdx] == null)
                {
                    data[rightChildIdx] = newData;

                    if (rightChildIdx > idxLast)
                        idxLast = rightChildIdx;

                    break;
                }
                else
                    currentIdx = rightChildIdx;
            }
            else
            {
                System.out.println(x: "Student with the same IPK already exists.");
                break;
            }
        }
    }

    public void traversePreOrder(int idxStart)
    {
        if (idxStart <= idxLast)
        {
            if (data[idxStart] != null)
            {
                data[idxStart].print();
                traversePreOrder(2 * idxStart + 1);
                traversePreOrder(2 * idxStart + 2);
            }
        }
    }
```

```
        System.out.println(x: "--- Array Tree Pre-Order Traversal ---");
        bta.traversePreOrder(idxStart: 0);
```

```
--- Array Tree Pre-Order Traversal ---
244107020138 - Devin - TI-1I - 3.57
244107020023 - Dewi - TI-1I - 3.85
244107020076 - Angelina - TI-1I - 3.54
244107020223 - Andhika - TI-1I - 3.72
244107020225 - Wahyu - TI-1I - 3.21
244107020226 - Bima - TI-1I - 3.37
244107020181 - Eiyu - TI-1I - 3.46
```

Github:
https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet14